Post-Quantum Cryptographic Standards for Pars Network

Pars Network Technical Whitepaper PNP-004

Cyrus Pahlavi, Pars Team research@pars.network Version 1.0 — February 2026

Abstract

We present the post-quantum cryptographic migration strategy for the Pars Network, addressing the "harvest now, decrypt later" threat relevant to diaspora communities under state surveillance. Adversaries may record encrypted traffic today for decryption by future quantum computers. We specify hybrid schemes combining classical primitives (X25519, Ed25519) with NIST-standardised post-quantum algorithms: ML-KEM (FIPS 203) for key encapsulation, ML-DSA (FIPS 204) for digital signatures, and SLH-DSA (FIPS 205) for stateless hash-based signatures. A phased migration plan maintains backward compatibility while strengthening all layers. Benchmarks on mobile hardware demonstrate that hybrid schemes add less than 3 ms to session establishment and less than 2 KB to signature sizes. We provide parameter selections, implementation guidance, and formal analysis of hybrid composition security under the assumption that at least one component remains secure.

Keywords: post-quantum cryptography, ML-KEM, ML-DSA, SLH-DSA, hybrid schemes, lattice-based, hash-based, migration strategy

Contents

1 Introduction **4**
1.1 Threat Timeline . . . 4
1.2 Contributions . . . 4

2 Background **4**
2.1 NIST Post-Quantum Standards . . . 4
2.2 Hardness Assumptions . . . 5
2.3 Current Cryptographic Inventory . . . 5

3 Algorithm Selection **5**
3.1 ML-KEM for Key Encapsulation . . . 5
3.2 ML-DSA for Digital Signatures . . . 5
3.3 SLH-DSA for Root-of-Trust . . . 6

4 Hybrid Composition Framework **6**
4.1 Design Principle . . . 6
4.2 Hybrid Key Encapsulation . . . 6
4.3 Hybrid Digital Signatures . . . 7

5 Migration Strategy **7**
5.1 Phased Approach . . . 7
5.2 Phase 0: Library Integration . . . 7
5.3 Phase 1: Hybrid Session Protocol . . . 8
5.4 Phase 2–3: Default and Mandatory . . . 8
5.5 Phase 4: PQ Primary . . . 8
5.6 L2 Account Migration . . . 8

6 Performance Analysis **9**
6.1 Operation Timing . . . 9
6.2 Size Overhead . . . 10
6.3 Session Establishment Impact . . . 10
6.4 L2 Transaction Size Impact . . . 10

7 Implementation Considerations **10**
7.1 Side-Channel Resistance . . . 10
7.2 Random Number Generation . . . 11
7.3 Key Storage . . . 11

8 Security Analysis **11**
8.1 Post-Migration Security Levels . . . 11
8.2 HNDL Mitigation . . . 11
8.3 Lattice Attack Landscape . . . 12
8.4 Diversity Argument for SLH-DSA . . . 12

9 Interoperability **12**
9.1 TLS 1.3 Integration . . . 12
9.2 Ethereum Address Compatibility . . . 12
9.3 Cross-Chain Bridge Signatures . . . 12

10 Formal Verification **13**
10.1 Implementation Verification . . . 13
10.2 Conformance Testing . . . 13

11 Conclusion **13**

A Algorithm OIDs **15**

B Hybrid KEM Test Vectors **15**

C Migration Checklist **15**

1 Introduction

Cryptographically relevant quantum computers (CRQCs) threaten the foundations of all blockchain and communication systems. Shor's algorithm [1] breaks RSA, DSA, and ECDH/ECDSA in polynomial time. Grover's algorithm [2] halves the effective security of symmetric ciphers and hash functions.

For diaspora communities under state surveillance, the "harvest now, decrypt later" (HNDL) strategy—where adversaries record encrypted communications for future quantum decryption—is a documented practice [3]. Traffic encrypted today with classical-only schemes may become readable within 10–20 years.

1.1 Threat Timeline

Table 1: Quantum Threat Timeline Estimates

Estimate	**Source**	**CRQC Year**
Optimistic	Mosca [4]	2030–2035
Median	NIST [5]	2035–2040
Conservative	NSA CNSA 2.0 [6]	2040–2050

Given 10–15 year confidentiality requirements and 5–10 year migration timelines, the window for action is now.

1.2 Contributions

- Concrete parameter selections for all Pars post-quantum primitives.
- Hybrid composition framework with formal security reduction.
- Phased migration preserving backward compatibility.
- Performance benchmarks on mobile and embedded hardware.
- Integration specifications for the Pars L2 (PNP-001), session protocol (PNP-003), and DID system (PNP-008).

2 Background

2.1 NIST Post-Quantum Standards

NIST finalised three standards in 2024 [7]:

1. **ML-KEM (FIPS 203):** Module-Lattice Key Encapsulation, based on CRYSTALS-Kyber [8].
2. **ML-DSA (FIPS 204):** Module-Lattice Digital Signature, based on CRYSTALS-Dilithium [9].
3. **SLH-DSA (FIPS 205):** Stateless Hash-Based Signatures, based on SPHINCS+ [10].

2.2 Hardness Assumptions

Definition 2.1 (Module-LWE)**.** *Given* $\mathbf{A} \in \mathbb{Z}_q^{k\times k}$ *and secret* $\mathbf{s} \in \mathbb{Z}_q^k$ *with small entries, distinguish* $(\mathbf{A}, \mathbf{As} + \mathbf{e})$ *from* $(\mathbf{A}, \mathbf{u})$ *where* $\mathbf{e}$ *is small error and* $\mathbf{u}$ *is uniform.*

Definition 2.2 (Module-SIS)**.** *Given* $\mathbf{A} \in \mathbb{Z}_q^{k\times \ell}$*, find non-zero* $\mathbf{z} \in \mathbb{Z}^\ell$ *with* $\|\mathbf{z}\|_\infty \leq \beta$ *and* $\mathbf{Az} = \mathbf{0} \bmod q$*.*

ML-KEM relies on M-LWE; ML-DSA on both M-LWE and M-SIS. SLH-DSA relies only on hash function security (second-preimage resistance).

2.3 Current Cryptographic Inventory

Table 2: Current Primitives and Quantum Vulnerability

Usage	Primitive	Classical	PQ Status
Key agreement	X25519	128-bit	Broken (Shor)
Signatures	Ed25519	128-bit	Broken (Shor)
Encryption	AES-256-GCM	256-bit	128-bit (Grover)
Hashing	SHA-256	256-bit	128-bit (Grover)
Sessions	X3DH + DR	128-bit	Broken (DH)
L2 accounts	secp256k1	128-bit	Broken (Shor)

3 Algorithm Selection

3.1 ML-KEM for Key Encapsulation

Table 3: ML-KEM Parameter Sets

Level	k	PK	CT	SS	NIST Lvl
ML-KEM-512	2	800 B	768 B	32 B	1
ML-KEM-768	3	1184 B	1088 B	32 B	3
ML-KEM-1024	4	1568 B	1568 B	32 B	5

Selection: ML-KEM-768. Level 3 matches 192-bit classical security. Key/ciphertext sizes below 1.2 KB. Operations under 0.5 ms on mobile ARM.

3.2 ML-DSA for Digital Signatures

Table 4: ML-DSA Parameter Sets

Level	PK	SK	Sig	NIST Lvl
ML-DSA-44	1312 B	2560 B	2420 B	2
ML-DSA-65	1952 B	4032 B	3309 B	3
ML-DSA-87	2592 B	4896 B	4627 B	5

Selection: ML-DSA-65. Level 3 matches ML-KEM-768. Signature (3.3 KB) acceptable for L2 transactions. Signing < 1 ms.

3.3 SLH-DSA for Root-of-Trust

Table 5: SLH-DSA Parameter Sets (Selected)

Variant	PK	SK	Sig	NIST Lvl
SLH-DSA-SHA2-128s	32 B	64 B	7856 B	1
SLH-DSA-SHA2-192f	48 B	96 B	35664 B	3
SLH-DSA-SHA2-256s	64 B	128 B	29792 B	5

Selection: SLH-DSA-SHA2-128s for genesis block, protocol parameters, and governance keys. Security rests solely on hash functions (insurance against lattice breakthroughs). Used infrequently, so larger signatures are acceptable.

4 Hybrid Composition Framework

4.1 Design Principle

Definition 4.1 (Hybrid Security)**.** *Scheme $\mathcal{H} = (\mathcal{C}, \mathcal{P})$ with classical $\mathcal{C}$ and post-quantum $\mathcal{P}$ satisfies:*

$$\mathrm{Adv}_{\mathcal{H}}^{\mathrm{IND}} \leq \min\big(\mathrm{Adv}_{\mathcal{C}}^{\mathrm{IND}},\ \mathrm{Adv}_{\mathcal{P}}^{\mathrm{IND}}\big) + \mathrm{negl}(\lambda).$$

4.2 Hybrid Key Encapsulation

Algorithm 1 Hybrid KEM: Encapsulate

Require: Classical PK pk_C (X25519), PQ PK pk_P (ML-KEM-768)
1: $(x, X) \leftarrow$ fresh X25519 pair
2: $\mathrm{ss}_C \leftarrow \mathrm{X25519}(x, \mathrm{pk}_C)$
3: $(\mathrm{ct}_P, \mathrm{ss}_P) \leftarrow \text{ML-KEM.Enc}(\mathrm{pk}_P)$
4: $\mathrm{SS} \leftarrow \mathrm{HKDF}\big(\mathrm{ss}_C \| \mathrm{ss}_P,\ X \| \mathrm{ct}_P,\ \texttt{"Pars-HybridKEM-v1"},\ 32\big)$
5: **return** $(X \| \mathrm{ct}_P,\ \mathrm{SS})$

Algorithm 2 Hybrid KEM: Decapsulate

Require: sk_C, sk_P, ciphertext (X, ct_P)
1: $\mathrm{ss}_C \leftarrow \mathrm{X25519}(\mathrm{sk}_C, X)$
2: $\mathrm{ss}_P \leftarrow \text{ML-KEM.Dec}(\mathrm{sk}_P, \mathrm{ct}_P)$
3: $\mathrm{SS} \leftarrow \mathrm{HKDF}\big(\mathrm{ss}_C \| \mathrm{ss}_P,\ X \| \mathrm{ct}_P,\ \texttt{"Pars-HybridKEM-v1"},\ 32\big)$
4: **return** SS

Theorem 4.1 (Hybrid KEM IND-CCA2)**.** *The hybrid KEM is IND-CCA2 secure if either X25519 (Gap-CDH) or ML-KEM-768 (M-LWE) is IND-CCA2 secure.*

Proof. The shared secret $\mathrm{SS} = \mathrm{HKDF}(\mathrm{ss}_C \| \mathrm{ss}_P, \ldots)$. By the HKDF extraction lemma [11], if either input is uniformly random and independent of the adversary's view, the output is computationally random. Under Gap-CDH, ss_C is random. Under M-LWE, ss_P is random. Since HKDF requires only one random component, the adversary's advantage is negligible unless both components are broken. □

4.3 Hybrid Digital Signatures

Definition 4.2 (Hybrid Signature)**.** *On message m:*

$$\sigma_H = \big(\text{Ed25519.Sign}(\text{sk}_C, m),\ \text{ML-DSA.Sign}(\text{sk}_P, m)\big).$$

Verification requires both components to pass.

Theorem 4.2 (Hybrid Signature EUF-CMA)**.** *The hybrid signature is EUF-CMA secure if either Ed25519 or ML-DSA-65 is EUF-CMA secure.*

Proof. A forger must produce both valid σ_C and σ_P. If Ed25519 is secure, σ_C cannot be forged. If ML-DSA is secure, σ_P cannot be forged. The forger succeeds only if both are broken simultaneously. □

5 Migration Strategy

5.1 Phased Approach

Table 6: Migration Phases

Phase	Timeline	Scope	Mode
0: Preparation	Q1 2026	Library integration	Classical only
1: Hybrid opt	Q2 2026	Session protocol	Hybrid (optional)
2: Hybrid dflt	Q3 2026	All new sessions	Hybrid (default)
3: Mandatory	Q1 2027	All connections	Hybrid (required)
4: PQ primary	2028+	Deprecate classical	PQ (with fallback)

5.2 Phase 0: Library Integration

Listing 1: PQ Crypto Interface

```go
type PQCrypto interface {
    KEMKeyGen() (KEMPub, KEMSec, error)
    KEMEncapsulate(pk KEMPub) (
        ct, ss []byte, err error,
    )
    KEMDecapsulate(
        sk KEMSec, ct []byte,
    ) ([]byte, error)

    DSAKeyGen() (DSAPub, DSASec, error)
    DSASign(sk DSASec, msg []byte) (
        []byte, error,
    )
    DSAVerify(
        pk DSAPub, msg, sig []byte,
    ) (bool, error)
}

type MLKEM768 struct{}

func (m *MLKEM768) KEMKeyGen() (
    KEMPub, KEMSec, error,
```

```go
) {
    pk, sk, err := mlkem768.GenerateKey()
    if err != nil {
        return nil, nil,
            fmt.Errorf("keygen: %w", err)
    }
    return pk, sk, nil
}

func (m *MLKEM768) KEMEncapsulate(
    pk KEMPub,
) ([]byte, []byte, error) {
    ct, ss, err := mlkem768.Encapsulate(pk)
    if err != nil {
        return nil, nil,
            fmt.Errorf("encap: %w", err)
    }
    return ct, ss, nil
}

func (m *MLKEM768) KEMDecapsulate(
    sk KEMSec, ct []byte,
) ([]byte, error) {
    ss, err := mlkem768.Decapsulate(sk, ct)
    if err != nil {
        return nil,
            fmt.Errorf("decap: %w", err)
    }
    return ss, nil
}
```

5.3 Phase 1: Hybrid Session Protocol

PSP session establishment (PNP-003) extended with hybrid X3DH. Prekey bundles include PQ public keys. Nodes advertise PQ capability via version flag.

5.4 Phase 2–3: Default and Mandatory

All new sessions default to hybrid (Phase 2). Classical-only rejected (Phase 3). All L2 transactions require hybrid signatures.

5.5 Phase 4: PQ Primary

Classical keys maintained only for external interoperability. Primary security rests on PQ algorithms.

5.6 L2 Account Migration

Listing 2: Hybrid Account Registry

```sol
contract HybridAccountRegistry {
    struct HybridAccount {
        address classicalAddr;
        bytes pqPubKey;        // ML-DSA-65
        bytes32 rootKeyHash;   // SLH-DSA
        uint8 phase;
```

```
        uint256 pqRegistered;
    }

    mapping(address => HybridAccount) public accts;

    function registerPQKey(
        bytes calldata pqPub,
        bytes calldata pqSig,
        bytes calldata classSig
    ) external {
        require(
            _verifyClassical(
                msg.sender, pqPub, classSig
            ), "Bad classical sig"
        );
        require(
            _verifyMLDSA(
                pqPub,
                abi.encodePacked(
                    msg.sender, pqPub
                ),
                pqSig
            ), "Bad PQ sig"
        );
        accts[msg.sender] = HybridAccount({
            classicalAddr: msg.sender,
            pqPubKey: pqPub,
            rootKeyHash: bytes32(0),
            phase: 1,
            pqRegistered: block.timestamp
        });
    }
}
```

6 Performance Analysis

6.1 Operation Timing

Table 7: PQ Operation Timing (mobile ARM)

Operation	**A17**	**SD 8g3**
ML-KEM-768 KeyGen	0.08 ms	0.12 ms
ML-KEM-768 Encapsulate	0.11 ms	0.16 ms
ML-KEM-768 Decapsulate	0.09 ms	0.14 ms
ML-DSA-65 KeyGen	0.15 ms	0.22 ms
ML-DSA-65 Sign	0.52 ms	0.71 ms
ML-DSA-65 Verify	0.18 ms	0.26 ms
SLH-DSA-SHA2-128s Sign	142 ms	198 ms
SLH-DSA-SHA2-128s Verify	6.2 ms	8.8 ms
Hybrid KEM	0.19 ms	0.28 ms
Hybrid Sig	0.57 ms	0.76 ms

6.2 Size Overhead

Table 8: Key and Signature Sizes

Scheme	PK	SK	Sig/CT
X25519 (classical)	32 B	32 B	32 B
ML-KEM-768	1184 B	2400 B	1088 B
Hybrid KEM	1216 B	2432 B	1120 B
Ed25519 (classical)	32 B	64 B	64 B
ML-DSA-65	1952 B	4032 B	3309 B
Hybrid Sig	1984 B	4096 B	3373 B
SLH-DSA-SHA2-128s	32 B	64 B	7856 B

6.3 Session Establishment Impact

Hybrid X3DH adds 1.3 ms over classical-only (2.1 ms vs 0.8 ms), negligible relative to network latency (50–500 ms).

6.4 L2 Transaction Size Impact

Table 9: Transaction Size Impact

Component	Classical	Hybrid
Addresses + value	72 B	72 B
Classical signature	65 B	65 B
PQ signature	—	3309 B
Total	**137 B**	**3446 B**

Mitigations: (1) ML-DSA batch aggregation via Fiat-Shamir; (2) $\approx 40\%$ zlib compression; (3) in rollup model, PQ sigs are verified by L2 validators, not posted to L1.

7 Implementation Considerations

7.1 Side-Channel Resistance

All PQ implementations must be constant-time:

Listing 3: Constant-Time Utilities

```go
func ctEqual(a, b []byte) bool {
    if len(a) != len(b) {
        return false
    }
    var acc byte
    for i := range a {
        acc |= a[i] ^ b[i]
    }
    return acc == 0
}

```

```go
func ctSelect(
    cond bool, a, b []byte,
) []byte {
    mask := byte(0)
    if cond {
        mask = 0xFF
    }
    out := make([]byte, len(a))
    for i := range a {
        out[i] = (a[i] & mask) |
                 (b[i] & ^mask)
    }
    return out
}
```

ML-KEM decapsulation uses implicit rejection (FO transform): on decryption failure, return a pseudorandom SS derived from secret key and ciphertext rather than an error, preventing chosen-ciphertext attacks.

7.2 Random Number Generation

Requirements:

1. OS CSPRNG (`/dev/urandom`, `SecRandomCopyBytes`).
2. Hardware RNG mixing where available.
3. NIST SP 800-90A DRBG seeded from OS entropy.

7.3 Key Storage

Table 10: Key Storage Requirements

Key Type	Size	Location
ML-KEM-768 pair	3.6 KB	Secure enclave
ML-DSA-65 pair	6.0 KB	Secure enclave
SLH-DSA root pair	0.2 KB	Hardware token
Prekey bundle	2.5 KB	Session directory
100 hybrid OTPKs	125 KB	Session directory

8 Security Analysis

8.1 Post-Migration Security Levels

8.2 HNDL Mitigation

Theorem 8.1 (HNDL Resistance)**.** *Traffic encrypted after Phase 1 resists HNDL if at the time of quantum decryption either (a) X25519 remains secure or (b) ML-KEM-768 remains secure.*

Proof. Direct from Theorem 4.1. □

Table 11: Security After Migration

Usage	Primitive	Classical	Quantum
Key agreement	Hybrid KEM	128-bit	≥128-bit
Tx signatures	Hybrid Sig	128-bit	≥128-bit
Root-of-trust	SLH-DSA-128s	128-bit	128-bit
Encryption	AES-256-GCM	256-bit	128-bit
Hashing	SHA-256	256-bit	128-bit

8.3 Lattice Attack Landscape

1. Current best lattice reduction: BKZ-β with $\beta \approx 400$. Breaking ML-KEM-768 requires $\beta > 800$ [13].
2. Known quantum speedups for lattice sieving are polynomial, not exponential [14].
3. Implementation-level side-channel attacks remain the primary practical concern.

8.4 Diversity Argument for SLH-DSA

SLH-DSA's security rests on hash function properties only, not lattice assumptions. If a polynomial-time lattice algorithm is discovered (breaking ML-KEM and ML-DSA), SLH-DSA-protected root keys remain secure. This provides cryptographic diversity across the trust hierarchy.

9 Interoperability

9.1 TLS 1.3 Integration

Listing 4: Hybrid TLS Configuration

```
cfg := &tls.Config{
    CurvePreferences: []tls.CurveID{
        tls.X25519MLKEM768,
        tls.X25519,
    },
    MinVersion: tls.VersionTLS13,
}
```

9.2 Ethereum Address Compatibility

Preserve 20-byte address format:

$$\text{addr} = \text{keccak256}\big(\text{pk}_C \| H(\text{pk}_P)\big)[12:]$$

9.3 Cross-Chain Bridge Signatures

Bridge contracts on Ethereum mainnet require hybrid verification:

Listing 5: Hybrid Verification Precompile

```
// Proposed precompile at 0x0110
// Input: pk_classical || pk_pq ||
//        sig_classical || sig_pq || msg
// Output: bool
function verifyHybrid(
```

```
    bytes calldata input
) external view returns (bool) {
    // Decode components
    (bytes memory pkC, bytes memory pkP,
     bytes memory sigC, bytes memory sigP,
     bytes memory msg) = abi.decode(
        input,
        (bytes,bytes,bytes,bytes,bytes)
    );
    // Both must verify
    return ecrecover_verify(pkC, msg, sigC)
        && mldsa_verify(pkP, msg, sigP);
}
```

10 Formal Verification

10.1 Implementation Verification

We target formal verification of:

1. **Constant-time execution:** Using ct-verif and Jasmin [24] to verify absence of secret-dependent branches and memory accesses.
2. **Functional correctness:** Proving decapsulation inverts encapsulation and signature verification accepts valid signatures, using the F* proof assistant.
3. **Protocol composition:** Verifying the hybrid composition preserves security using CryptoVerif [25].

10.2 Conformance Testing

All implementations must pass:

1. NIST Known-Answer Test (KAT) vectors for ML-KEM-768, ML-DSA-65, and SLH-DSA-SHA2-128s.
2. Pars-specific hybrid KEM and hybrid signature test vectors.
3. Interoperability tests against reference implementations.

11 Conclusion

The Pars Network post-quantum migration addresses the immediate HNDL threat through hybrid schemes composing classical and PQ primitives. The phased migration preserves compatibility while strengthening all layers. Hybrid operations add less than 3 ms to session establishment. SLH-DSA provides insurance against lattice breakthroughs for root-of-trust operations. Mandatory hybrid mode is targeted for Q1 2027, with PQ-primary for 2028 and beyond.

References

[1] P. Shor, "Algorithms for quantum computation," *FOCS*, pp. 124–134, 1994.

[2] L. Grover, "A fast quantum mechanical algorithm for database search," *STOC*, pp. 212–219, 1996.

[3] NSA, "Commercial National Security Algorithm Suite 2.0," 2022.

[4] M. Mosca, "Cybersecurity in an Era with Quantum Computers," *IEEE Security & Privacy*, 16(5), 2018.

[5] NIST, "Post-Quantum Cryptography: Call for Proposals," 2016.

[6] NSA, "CNSA 2.0," 2022.

[7] NIST, "FIPS 203, 204, 205," 2024.

[8] J. Bos *et al.*, "CRYSTALS – Kyber," *Euro S&P*, 2018.

[9] L. Ducas *et al.*, "CRYSTALS – Dilithium," *TCHES*, 2018(1), 2018.

[10] D. J. Bernstein *et al.*, "SPHINCS+," *CHES*, 2019.

[11] H. Krawczyk, "HKDF," *CRYPTO 2010*, pp. 631–648, 2010.

[12] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid Key Exchange in TLS 1.3," *IETF Draft*, 2022.

[13] M. Albrecht *et al.*, "Estimate All the LWE, NTRU Schemes!," *SCN 2018*, 2018.

[14] T. Laarhoven, M. Mosca, and J. van de Pol, "Finding shortest lattice vectors faster using quantum search," *Des. Codes Cryptogr.*, 77, 2015.

[15] D. J. Bernstein, "Curve25519," *PKC 2006*, pp. 207–228, 2006.

[16] D. J. Bernstein *et al.*, "High-speed high-security signatures," *J. Crypt. Eng.*, 2(2), 2012.

[17] D. McGrew and J. Viega, "GCM," *NIST*, 2004.

[18] C. Peikert, "A Decade of Lattice Cryptography," *FnTTCS*, 10(4), 2016.

[19] O. Regev, "On lattices, learning with errors," *STOC*, pp. 84–93, 2005.

[20] M. Ajtai, "Generating hard instances of lattice problems," *STOC*, pp. 99–108, 1996.

[21] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *EUROCRYPT*, 2010.

[22] F. Giacon, F. Heuer, and B. Poettering, "KEM Combiners," *PKC 2018*, pp. 190–218, 2018.

[23] N. Bindel *et al.*, "Hybrid Key Encapsulation Mechanisms," *PQCrypto 2019*, 2019.

[24] J. B. Almeida *et al.*, "Jasmin: High-Assurance and High-Speed Cryptography," *CCS*, 2017.

[25] B. Blanchet, "CryptoVerif: Computationally Sound Mechanized Prover," *Dagstuhl Seminar*, 2008.

[26] G. Wood, "Ethereum Yellow Paper," 2014.

Algorithm	OID
ML-KEM-768	2.16.840.1.101.3.4.4.2
ML-DSA-65	2.16.840.1.101.3.4.3.18
SLH-DSA-SHA2-128s	2.16.840.1.101.3.4.3.20

A Algorithm OIDs

B Hybrid KEM Test Vectors

Listing 6: Test Vector Structure

```
type HybridKEMTestVector struct {
    ClassicalSK  [32]byte  // X25519
    ClassicalPK  [32]byte
    PQSK         []byte    // ML-KEM-768
    PQPK         []byte
    Ciphertext   []byte    // 1120 bytes
    SharedSecret [32]byte
}

// Full vectors at:
// github.com/pars-network/pqc-test-vectors
```

C Migration Checklist

1. Integrate ML-KEM-768 and ML-DSA-65 libraries
2. Add hybrid key pair generation to wallet software
3. Extend prekey bundles with PQ components
4. Deploy hybrid X3DH in session protocol
5. Add PQ key registration to L2 account registry
6. Update bridge contracts for hybrid verification
7. Enable SLH-DSA for governance key ceremonies
8. Run interoperability tests across all clients
9. Publish migration documentation for third-party developers
10. Conduct security audit of PQ implementations